Case 10: Negative Testing — Invalid Inputs

Intermediate+

Verify the API handles invalid or edge-case inputs correctly.

New concept: Negative testing + pytest.mark.parametrize with xfail

API Endpoint

```
GET https://jsonplaceholder.typicode.com/posts/99999
```

What You Will Learn

- Negative testing — verifying failure cases, not just happy paths
- Testing non-existent resources (404)
- Testing edge cases: id=0, negative id, very large id
- Combining parametrize with pytest.param and xfail marks

Study Plan

Step 1 — What is negative testing?

Positive tests verify things work. Negative tests verify the API handles bad inputs gracefully — wrong IDs, missing fields, invalid types. Both are essential in real automation roles.

Step 2 — Test a non-existent resource

Call GET /posts/99999. A well-behaved API returns 404 Not Found. Assert response.status_code == 404.

Step 3 — Edge case IDs with parametrize

Use parametrize to test ids 99999, 0, -1 all in one test function. Think: what is the parametrize variable here — the full URL or just the id?

Step 4 — pytest.param with marks

Inside parametrize, use pytest.param(value, marks=pytest.mark.xfail) to mark specific cases as expected failures if the API behavior is uncertain.

Step 5 — Think about the loop

There is no explicit for loop here — parametrize IS the loop. But inside the test, think: do you need to loop over anything in the response, or is one assert enough?

Think Before You Code

Ask yourself these questions before writing a single line:

➤ What is the changing value between each test run — the full URL or just the id?

➤ For a 404 response, does response.json() return a dict or a list? Do you need a loop?

➤ What happens if you call response.json() on a 404 — will it always have a body?

➤ How is parametrize acting like a loop here — what does each 'iteration' represent?

Your Task

Write a parametrized pytest test that sends GET requests with invalid post IDs.

Test these IDs: 99999, 0, -1, 99999999

Before writing, think:
- What is the parametrize variable — the id or the full URL?
- Do you need a for loop inside the test, or is parametrize enough?
- What type does the response return for a 404?

The test must verify:
1. Status code is 404 for each invalid id
2. Use @pytest.mark.parametrize to test all IDs in one function

BONUS 1: Add a valid id (e.g. 1) to the parametrize list and mark it
with xfail — because we expect 200, not 404

BONUS 2: Print the status code for each ID to see what JSONPlaceholder
actually returns for id=0 and id=-1 (it may surprise you!)

> **Hint:** The parametrize variable is the id. Build the URL inside the test with an f-string. No for loop needed inside — parametrize handles the repetition.

Base URL: https://jsonplaceholder.typicode.com | No auth required | Level 2 — builds on Cases 1-7